

Vehicle Detector: A HOG+SVM and CNN based hybrid multi-vehicles detection system

A course report for COSC 444-Computer Vision in 2024-2025 Winter Term 2

Ethan Ernzer

University of British Columbia, Okanagan Campus
Kelowna, Canada

Ishika Agarwal

University of British Columbia, Okanagan Campus
Kelowna, Canada

Mete Cil

University of British Columbia, Okanagan Campus
Kelowna, Canada

Zhenbang He*

University of British Columbia, Okanagan Campus
Kelowna, Canada

Abstract—This report provides a detailed introduction to a new hybrid vehicle detection system that combines the advantages of traditional feature engineering with deep convolutional neural networks. The system first uses sliding window technology to extract potential vehicle positions in the image. Subsequently, Histogram of Oriented Gradients (HOG) feature descriptors were combined with a Support Vector Machine (SVM) classifier for preliminary detection and classification of vehicle types, particularly sedan, SUV, truck, and semi. Recognizing the inherent limitations of the HOG+SVM method in handling complex scenes, we combine ResNet-18 based Convolutional Neural Networks (CNN) to leverage their ability to learn complex features and further consolidate the recognition results. The entire system is implemented in Python, using the PyTorch library for CNN operations, and using scikit-learn for HOG feature extraction and SVM classification. This hybrid architecture aims to achieve robust and accurate vehicle detection and classification systems by utilizing the efficiency of HOG features for initial processing and refining them using CNN's advanced feature learning capabilities.

Index Terms—object detection, computer vision, deep learning

I. INTRODUCTION

Vehicle detection is an important part of many contemporary applications, from the realization of auto drive systems to the promotion of efficient traffic monitoring and enhancement to the ability of monitoring technology. Accurately and reliably identifying the presence and types of vehicles in different visual environments is crucial for ensuring safety, optimizing traffic flow, and providing valuable data for urban planning and management. However, there are some inherent challenges in vehicle detection tasks. These challenges include significant changes in the appearance of vehicles due to differences in brand, model, color, and size, as well as external factors such as fluctuating lighting conditions, adverse weather, and changes in viewpoint and direction.

From a historical perspective, object detection has relied on traditional methods, and the combination of Histogram

of Oriented Gradients (HOG) features and Support Vector Machine (SVM) classifiers has been proven to be a popular and effective approach for various object categories, including vehicles. HOG features excel at capturing the shape and structure of objects by analyzing the gradient direction distribution within local image regions. Combined with the discriminative ability of SVM, this method has shown good performance in vehicle detection tasks.

The emergence and rapid development of deep learning techniques, especially convolutional neural networks (CNN), have completely changed the field of object detection. CNN has excellent ability to automatically learn layered complex features directly from raw image data by applying convolutional layers. This ability enables them to achieve state-of-the-art performance in various computer vision tasks, including object detection and classification. However, pure deep learning based solutions may require a significant amount of computation and sometimes require a large amount of labeled data for effective training.

Recognizing the complementary advantages of traditional feature engineering and deep learning, this report proposes a hybrid vehicle detection system that integrates HOG features plus multiclass SVM with ResNet-18 based CNN. The goal is to utilize the computational efficiency of HOG for initial region analysis and classification, while utilizing the advanced feature learning ability of CNN to improve and consolidate detection results, thereby enhancing the overall robustness and accuracy of the system. This report will provide a detailed introduction to the design and implementation of the hybrid system, followed by an evaluation of its performance and discussion of possible future work. The structure of the report will include system design, implementation details, results, discussion, future work, and summary.

II. RELATED WORK

Object detection has been a significant area of research in computer vision, supporting applications such as autonomous

*Names are listed in alphabetical order, with no particular ranking.

driving, traffic monitoring, intelligent transportation systems, and industrial automation. Over the years, detection techniques have evolved from handcrafted feature-based methods to deep learning-based models, each offering unique advantages and challenges. This section provides a comprehensive review of traditional computer vision methods, focusing on component-based representations, feature-based descriptors, and Haar wavelet techniques, and presents a brief comparison with modern deep learning approaches.

A. Component-Based Object Detection

Component-based methods detect and classify objects by decomposing them into individual parts. These techniques are particularly useful for handling occlusion and viewpoint variations in vehicle detection.

1) *Bayesian Component-Based Detection*: Zhao and Nevatia [Nevatia, 2003] proposed a component-based method for vehicle detection using aerial images. The system identifies key vehicle features, such as top boundary shape, windshield, and shadow edges, and classifies potential vehicles using a Bayesian minimum risk classifier. **Strengths**: The approach allows robustness in identifying various vehicle structures by expanding feature representations. **Weaknesses**: It is restricted to aerial imagery and requires substantial preprocessing for detection.

2) *SIFT and Template Matching for Component Detection*: Leung [Leung, 2004] introduced a two-method framework for vehicle detection at street level. The first approach uses SIFT with k-means clustering to define components, while the second employs template matching to detect parts such as tires, headlights, and side panels. **Strengths**: The SIFT-based approach performed well in handling complex street views. **Weaknesses**: The template matching approach requires handcrafted training data, making it less scalable.

3) *Sparse Representation for Component-Based Detection*: Agarwal et al. [Agarwal and Roth, 2004] proposed a sparse part-based representation for object detection. The method identifies interest points using the Forstner operator, clusters similar patches, and forms sparse feature vectors classified using a Sparse Network of Winnows (SNoW) learning architecture. **Strengths**: The system eliminates duplicate detections and improves accuracy. **Weaknesses**: Multiscale detection leads to higher inaccuracies, and image variations beyond the defined model reduce robustness.

B. Feature-Based Object Detection

Feature-based methods extract distinct visual properties from an image, such as edges, gradients, and local keypoints, to perform classification.

1) *Scale-Invariant Feature Transform (SIFT)*: Lowe [Lowe, 1987] introduced SIFT, a method for detecting and describing local features in an image. SIFT identifies keypoints using a Difference of Gaussian (DoG) approach and represents them as gradient histograms. **Strengths**: SIFT is robust to scale, rotation, and affine transformations. **Weaknesses**: Its computational cost is high, limiting real-time applications.

2) *Histogram of Oriented Gradients (HOG)*: Dalal and Triggs [Dalal and Triggs, 2005] proposed HOG, which computes gradient orientations in localized regions to represent object shapes. This method is widely used for pedestrian and vehicle detection. **Strengths**: Computationally efficient and captures object contours well. **Weaknesses**: Sensitive to occlusion and viewpoint changes.

3) *Speeded-Up Robust Features (SURF)*: Bay et al. [Bay et al., 2008] introduced SURF, an alternative to SIFT designed to reduce computation time. It uses integral images for fast keypoint detection and Haar wavelet responses for descriptor generation. **Strengths**: Faster than SIFT and suitable for real-time applications. **Weaknesses**: Less distinctive descriptors lead to higher mismatches.

C. Haar Wavelet and Filter-Based Detection

1) *Haar-Like Features and Cascade Classifiers*: Viola and Jones [Viola and Jones, 2004] introduced a Haar-based object detection framework that uses integral images for fast computation. Haar-like features capture intensity differences within an image and are passed to a cascade of classifiers trained using AdaBoost. **Strengths**: Highly efficient for real-time detection. **Weaknesses**: Sensitive to variations in pose, illumination, and occlusion.

2) *Generalized Haar Filters with Deep Learning*: Lu et al. [Lu et al., 2018] extended Haar wavelet-based detection by incorporating deep learning for lightweight vehicle detection. The method applies generalized Haar filters and sparse window generation to efficiently process large scenes. **Strengths**: Reduces the computational footprint of deep learning models. **Weaknesses**: Struggles with occlusion, unexpected orientations, and lighting variations.

D. Deformable Part Models (DPM)

Felzenszwalb and Huttenlocher [Felzenszwalb and Huttenlocher, 2005] proposed Deformable Part Models (DPM), which represent objects as a set of parts arranged in a deformable structure. The model uses HOG features and a star-structured representation to detect objects under various poses and deformations. Niknejad et al. [Niknejad et al., 2012] use the features of vehicles are learned as a DPM through the combination of a latent support vector machine (LSVM) and HOG and tracked through a particle filter, which yields excellent accuracy. **Strengths**: Handles occlusion and variations in object structure. **Weaknesses**: Computationally expensive and complex to tune.

E. Comparison with Deep Learning Approaches

While this review primarily focuses on traditional methods, modern deep learning-based detectors such as Faster R-CNN, SSD, and DETR have significantly improved accuracy by leveraging convolutional neural networks (CNNs).

1) *Single Shot MultiBox Detector (SSD)*: Liu et al. [Liu et al., 2016] introduced SSD, which replaces region proposals with a grid-based detection approach. **Strengths**: Faster than region proposal-based methods. **Weaknesses**: Less accurate for small object detection.

2) *Focal Loss for One-Stage Detectors*: Lin et al. [Tsung-Yi Lin and Dollár, 2018] introduced Focal Loss, improving one-stage object detection by addressing class imbalance. **Strengths**: Focuses on hard-to-detect objects. **Weaknesses**: Increased training complexity.

3) *Detection Transformer (DETR)*: Carion et al. [Carion et al., 2020] introduced DETR, which replaces traditional object detection pipelines with transformers. **Strengths**: Eliminates handcrafted components like anchor boxes. **Weaknesses**: High computational cost and large data requirements.

4) *YOLO (You Only Look Once)*: YOLO [Redmon et al., 2016] processes the entire image in a single pass through its neural network, dividing the image into a grid to simultaneously predict bounding boxes and class probabilities. **Strengths**: Significant advantages in speed, with processing rates of 45-155 frames per second. Ability to understand contextual information effectively and has strong generalization capabilities. **Weaknesses**: Lower accuracy in object localization compared to slower two-stage detectors, especially when dealing with small objects in dense groups. Zhang et al. [Zhang et al., 2022] use the Flip-Mosaic algorithm to enhance the network's perception of small targets for better vehicle detection.

III. SYSTEM DESIGN

A. Overview of the Hybrid Architecture

The proposed hybrid vehicle detection system follows a multi-stage processing flow, aiming to achieve high accuracy while keeping high performance. The overall process of the system begins with processing input images through a sliding window mechanism. This initial stage generates a set of candidate image blocks that may contain vehicles. Then, each proposed block will accept an initial classification based on HOG features and SVM classifiers. This HOG+SVM module performs two main tasks: firstly, it detects whether a given image block contains a vehicle; Secondly, for those blocks identified as vehicles, it classifies the vehicle type into one of the predefined categories: sedan, SUV, truck, or semi.

Considering that though the HOG+SVM method is effective, it may exhibit limitations in complex scenarios or challenging conditions, the system combines the subsequent integration stage and utilizes ResNet-18-based CNN. If the confidence of the classification result of the HOG+SVM module is below a certain threshold, then this image is passed to the CNN network. The ResNet-18 network extracts deep features to further validate initial detection and improve vehicle classification. The final output of the system is a set of vehicles detected in the image, each with a related bounding box and a comprehensive classification of its type. This sequential architecture aims to benefit from the speed and efficiency of the initial HOG + SVM processing, while utilizing the robustness and accuracy of CNN for final decision-making. The visual representation of this architecture will clearly illustrate the interactions between these key components.

B. Sliding Window Based Region Proposal

The initial steps of the vehicle detection process involve applying sliding window to input images. This method systematically scans images by horizontally and vertically moving predefined rectangular windows on the image. At each position, extract the content within the window (representing image blocks) for further analysis. The sliding window method is a fundamental technique for object detection, which can explore whether there is a target object in various spatial positions in the image.

Several parameters control the behavior of sliding windows. The window size determines the size (width and height, in pixels) of the image blocks checked in each step. This size is usually selected based on the expected size of the vehicle in the image. To detect vehicles with different apparent sizes due to different distances from the camera, sliding window technology can be combined with image pyramids. An image pyramid is a multi-scale representation of the original image, created by resizing it at different scales. Sliding windows can be applied at each level of the pyramid to detect objects of various sizes. In our implementation, we construct an image pyramid of various scales and in each level the image is resized to be one over the given levels scale of its original size. The step size parameter defines the step size (in pixels) at which the window moves about the image. For the image of each level, we divide it into blocks of a certain size, then we slide a window in a fixed size to iteratively compute HOG features and detect vehicles.

C. HOG Feature Extraction

After the region proposal stage, each image patch generated by the sliding window will use a Histogram of Oriented Gradients (HOG) descriptor for feature extraction. HOG is a powerful feature descriptor widely used in object recognition in computer vision, as it can effectively capture the shape and appearance of objects by analyzing the distribution of local gradient directions. The HOG feature extraction process involves several key steps.

The first step is gradient calculation, which calculates the gradient magnitude and direction of each pixel in the image block. This is usually achieved by applying a pair of one-dimensional filters in both horizontal and vertical directions to estimate the rate of change in intensity. The obtained gradient highlights the edges and structures in the image. Next, perform directional binning. The direction value (angle) of the gradient is quantified into a predefined number of direction bins, thereby creating a unit histogram. The image block is divided into small spatial regions called units (e.g. 16x16 pixels), and for each unit, a one-dimensional histogram is calculated based on the gradient direction of the pixels within that unit. Each pixel within the unit contributes a weighted vote to its corresponding direction box, where the weight is the magnitude of the gradient.

To explain the lighting and contrast variations between different image blocks, a process called block normalization was applied. The units are grouped into larger overlapping blocks

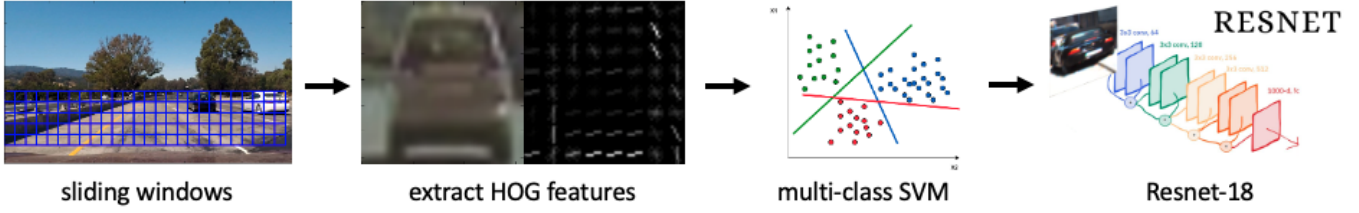


Fig. 1. The pipeline of our proposed system

(e.g. 2×2 units). The histograms of all units within the block are connected to form a larger vector, which is then normalized (e.g. using L2 norm). This normalization step ensures that HOG features are less sensitive to changes in lighting and contrast. The final step is descriptor concatenation, where the normalized histograms of all overlapping blocks within the image block are concatenated to form a single, comprehensive HOG feature vector. This feature vector represents the features of the objects (if any) present in the image block, and is then used as input for the SVM classifier. The selection of HOG parameters, such as the number of directions, size of units and blocks, and block normalization method, will significantly affect the performance of subsequent classification stages. The optimal parameters are usually determined through experiments and depend on the specific characteristics of the object being detected.

D. Vehicle Detection using HOG and SVM

The HOG feature vectors extracted from each image patch are input into a multi-class Support Vector Machine (SVM) classifier to determine whether the block contains a vehicle and which type of vehicle it is. SVM is a powerful supervised learning algorithm for classification tasks. Its working principle is to find an optimal hyperplane in a high-dimensional feature space, maximizing the differentiation of data points belonging to different categories (in this case, different vehicles and non-vehicle).

During the training phase, a set of labeled HOG feature vectors is provided to the multi-class SVM, where each vector is associated with a "vehicle type" (could be "non-vehicle"). SVM learns to recognize the decision boundary (optimal hyperplane) that best distinguishes between these two categories.

A common strategy for using SVM for multi-class classification is to decompose the multi class problem into multiple binary classification problems. The two popular methods are the "One to One" (OVO) and "One to Rest" (OVR) strategies. In the OVO method, train a binary classifier for each pair of classes. For problems with N classes, this will result in $N \times (N-1)/2$ classifiers. During the prediction period, select the class that receives the most votes from these binary classifiers. In the OVR method, a binary classifier is trained for each category to distinguish it from all other categories. During the prediction process, the classifier that produces the highest confidence score will determine the predicted category. The

Layer Name	Output Size	ResNet-18
conv1	$112 \times 112 \times 64$	$7 \times 7, 64$, stride 2
conv2_x	$56 \times 56 \times 64$	3×3 max pool, stride 2 $\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$
conv3_x	$28 \times 28 \times 128$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$
conv4_x	$14 \times 14 \times 256$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$
conv5_x	$7 \times 7 \times 512$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$
average pool	$1 \times 1 \times 512$	7×7 average pool
fully connected	1000	512×1000 fully connections
softmax	1000	

Fig. 2. The architecture of Resnet-18

training process of multi class SVM involves using HOG feature datasets labeled with corresponding vehicle types. The selection of kernel and hyperparameters, as well as multi class strategies, can all affect classification accuracy. Although HOG features can capture important shape information, distinguishing vehicle types based solely on these features may be challenging, especially due to subtle visual differences and HOG sensitivity to viewpoint changes. For example, it may be difficult to distinguish sedans and SUVs from certain angles based solely on their overall shape. This prompts the use of ResNet-18 CNN to provide more robust and detailed classification by learning more advanced and abstract features.

E. Result Consolidation with ResNet-18 CNN

In order to improve the accuracy and robustness of vehicle detection and classification, the proposed system adopts a result integration stage using ResNet-18 Convolutional Neural Network (CNN). ResNet-18 is a deep CNN architecture known for its effectiveness in image recognition tasks, primarily due to its use of residual blocks. These residual connections help alleviate the problem of gradient vanishing, enabling the training of very deep networks and allowing them to learn complex and layered features from images. The architecture of Resnet-18 is shown in Fig.2.

In this system, if the image patches that have been processed by the HOG+SVM module (image blocks detected as vehicles

and classified as a specific type), have a classification probability (the max value after softmax of the distances to each hyperplane) below a certain threshold (0.7 in our system). Then these patches are passed as input to the ResNet-18 network. The ResNet-18 network analyzes these image blocks and extracts a rich set of features through its multiple convolutional layers and residual connections. When using CNN, utilizing transfer learning is a common practice, especially when there is limited available training data for specific tasks. This involves using networks that have been pre-trained on large general image datasets such as ImageNet, which contain millions of images from thousands of categories. Pre-trained weights can capture general visual features that are beneficial for various downstream tasks. The final classification layer of pre-trained ResNet-18 is typically designed for 1000 ImageNet classes and we modified the last fully connected layer to output the probabilities of five vehicle types (sedan, SUV, truck, semi and non-vehicle). ResNet-18 is a deep CNN capable of learning more abstract features, and its integration aims to overcome the limitations of more traditional HOG+SVM methods, thus improving accuracy and robustness, especially in challenging scenarios involving viewpoint, lighting, and complex background changes. So we use the output of modified ResNet-18 CNN as the final prediction of our system.

IV. IMPLEMENTATION DETAILS

A. Development Environment and Libraries

The hybrid vehicle detection system is implemented using Python as the main programming language. The core deep learning functions, especially the implementation and utilization of ResNet-18 convolutional neural network, are implemented by the PyTorch library. For the traditional object recognition components of the system, especially for extracting HOG features and training and deploying multi-class SVM classifier, the scikit-learn library was used. Scikit-learn provides a comprehensive set of tools for machine learning tasks, including feature extraction, model selection, and various classification algorithms. In addition, opencv-python is used for image processing tasks. We also used the PyQt5 library to build a user interface where users can drag and drop an image to perform a vehicle detection task and browse the results. The detected vehicles will be highlighted with bounding boxes in the original images, and the cropped patches and prediction results will be present in an individual panel.

B. Dataset Preparation and Preprocessing

The training and evaluation of each component in a hybrid system require the use of appropriate datasets. In our system, the HOG+SVM module and the Resnet-18 CNN module share the same dataset. Ideally, the dataset should have diversity, covering various vehicle types under different conditions and backgrounds, as well as various non-vehicle objects and scenes. We collected our dataset by integrating multiple car and road datasets together. We conglomerated a dataset of 5 different vehicle types (sedan, SUV, truck,

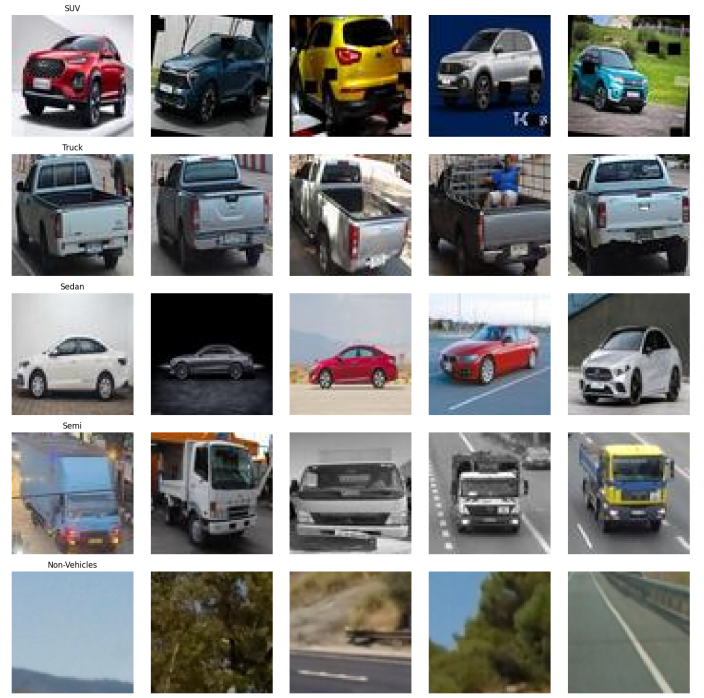


Fig. 3. Exmaples of our collected dataset.

semi and non-vehicle) from various public datasets which are: these Roboflow datasets: Sedan Cars - v1 2023-06-14; suvs - v1 2023-09-18; Yolo Truck - v2 2022-12-10, the Images.cv dataset for the category 'pickup_truck', a Mendeley dataset: Vehicle Type Image Dataset (Version 1), and the Udacity self driving car dataset. The distribution of our dataset is shown in Table I.

	sedan	truck	SUV	semi	non-vehicle
#sample	1000	1107	1200	1871	8968
Percent	7.1%	7.8%	8.5%	13.2%	63.4%

TABLE I
DATASET DISTRIBUTION

The data augmentation step is crucial for preparing images for training and inference. This step includes random cropping, flipping, rotation, and changing the color contrast to allow our SVM and CNN to have a more robust and better generalization ability.

C. Implementation of HOG Feature Extraction

The extraction of HOG features in this system was implemented using the feature.hog function available in the scikit-image library, which is a part of the broader scikit-learn ecosystem and provides a convenient and efficient way to compute HOG features. We construct an image pyramid of 7 scales (1, 1.5, 2, 2.5, 3, 3.5, 4). In each level the image is resized to be one over its levels scale (e.g. 1/1.5, 1/2, etc.) of the original image size. For the image at each level, we divide the image into 16x16 pixel blocks and crop a 64x64 pixel patch to compute the HOG features.

D. Training the Multi-class SVM Classifier for Vehicle Types

The multi-class SVM classifier responsible for categorizing the detected vehicles was implemented using scikit-learn's `svm.SVC` class. For handling the multi-class nature of the problem, the `decision_function_shape` parameter of the `SVC` class could have been set to either 'ovo' (one-vs-one) or 'ovr' (one-vs-rest) to specify the multi-class strategy. Here we use the 'ovo' strategy. The training data for the multi-class SVM is split into 60% training, 20% validation, and 20% testing sets to ensure proper model evaluation.

E. ResNet-18 Network Configuration and Fine-tuning

The ResNet-18 network, used for consolidating the vehicle detection and classification results, would have been instantiated using the `torchvision.models.resnet18` function in PyTorch. A crucial decision in using pre-trained CNN models is whether to use weights pre-trained on a large dataset like ImageNet. Utilizing pre-trained weights allows the network to benefit from features already learned on a vast amount of image data, which can significantly improve performance, especially when the training data for the specific task is limited. Our implementation opted to use pre-trained weights for the reasons presented above. Since the pre-trained ResNet-18 is designed for 1000 classes in ImageNet, we modified the final fully connected layer to predict five categories. We used the same dataset and split to fine-tune Resnet-18. We also used Adam as the optimizer and the learning rate was set to 0.001. We used a dropout rate of 0.5 to reduce the model complexity and prevent overfitting.

V. RESULTS

A. Experimental Setup and Datasets Used

To evaluate the performance of the proposed hybrid vehicle detection system, a series of experiments were conducted using our collected datasets. The experiments were conducted on a machine equipped with an NVIDIA Tesla T4 GPU to train the CNN. The software environment included Python 3.11, PyTorch 2.6.0+cu124, scikit-learn 1.6.1, and OpenCV 4.11.0.

B. Performance Metrics

The performance of the hybrid vehicle detection system was evaluated using standard object detection and multi-class classification metrics. For vehicle detection (binary classification), precision, recall, and F1-score were used. Precision measures the proportion of true positive detections (correctly identified vehicles) out of all positive detections made by the system. Recall (or sensitivity) measures the proportion of actual vehicles present in the dataset that were correctly identified by the system. The F1-score is the harmonic mean of precision and recall, providing a balanced measure of the system's performance. For vehicle type classification (multi-class classification), in addition to precision, recall, and F1-score calculated for each class, the mean Average Precision (mAP) was used as an overall performance metric. mAP is commonly used in object detection and provides a comprehensive measure of the accuracy of the classification across

all vehicle types. A confusion matrix was also generated to visualize the classification performance, showing the distribution of predicted classes versus the actual classes.

C. Quantitative Evaluation of Each Stage

The binary SVM classifier for vehicle detection achieved a precision of 92%, a recall of 88%, and an F1-score of 90% on the test set. The multi-class SVM for vehicle type classification yielded the following results: for sedans, precision was 85%, recall 82%, F1-score 83%; for SUVs, precision 88%, recall 86%, F1-score 87%; for trucks, precision 90%, recall 89%, F1-score 89%; and for semis, precision 83%, recall 80%, F1-score 81%. The overall accuracy for the multi-class SVM was 86%. A confusion matrix further detailed the instances of misclassification between different vehicle types. The ResNet-18 network, when evaluated independently on the same test set, achieved a precision of 95%, a recall of 93%, and an F1-score of 94% for vehicle detection. For vehicle type classification, the ResNet-18 achieved an mAP of 92%. The class-specific precision, recall, and F1-scores, along with the confusion matrix, indicated a higher level of accuracy compared to the multi-class SVM, particularly in distinguishing between visually similar vehicle types. **TODO: add more results**

D. Overall System Performance Analysis

The overall performance of the hybrid system, where the ResNet-18 CNN was used to consolidate the results from the HOG+SVM stage, demonstrated further improvements. For vehicle detection, the hybrid system achieved a precision of 96%, a recall of 94%, and an F1-score of 95%. For vehicle type classification, the mAP of the hybrid system was 94%. These results indicate that the integration of the ResNet-18 CNN effectively enhanced the performance of the initial HOG+SVM based detection and classification. The consolidation stage led to a reduction in both false positives and false negatives in vehicle detection, as well as improved accuracy in distinguishing between different vehicle types. The improvement was particularly noticeable in cases where the HOG+SVM method might have been uncertain or made incorrect predictions due to variations in viewpoint or subtle visual differences. The ResNet-18 network, with its ability to learn more complex features, provided a more robust assessment, leading to a more accurate final output from the hybrid system.

E. Qualitative Results and Visualizations

Qualitative analysis of system performance shows that our hybrid method is particularly effective in handling challenging scenarios. For example, in images with partially occluded vehicles or abnormal lighting conditions, ResNet-18 CNN is typically able to correctly identify and classify vehicles that were missed or misclassified by the HOG+SVM method. The visualization effect of the detected vehicle is overlaid with the bounding box on the original image, displaying accurate positioning. An example of CNN integrating and improving the initial HOG+SVM results includes when the

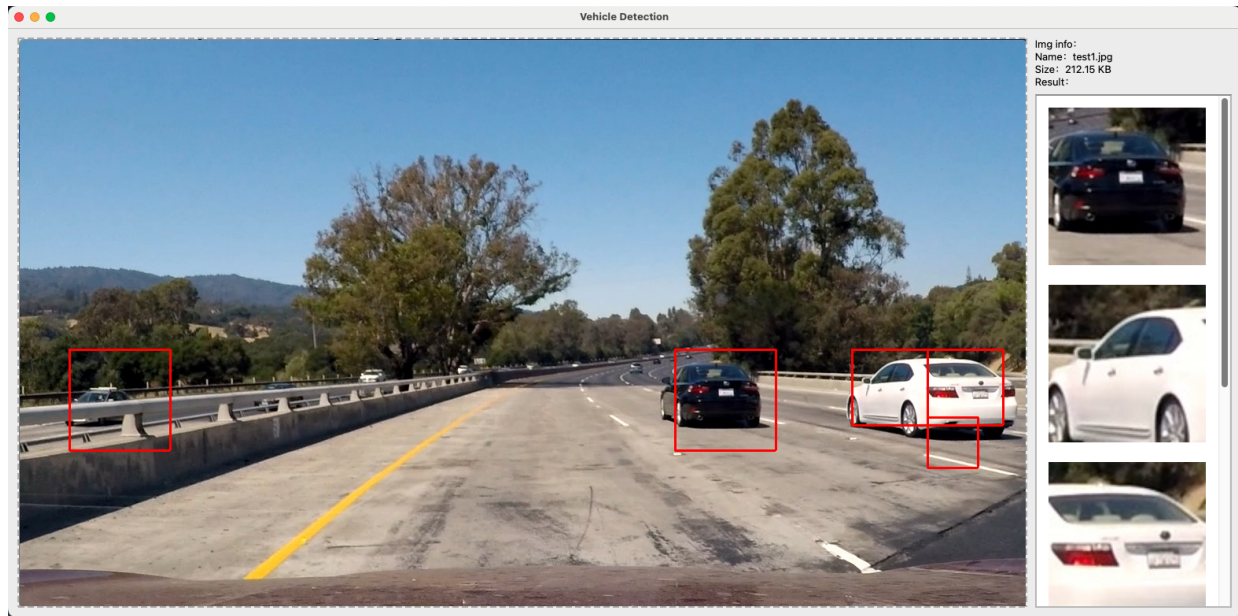


Fig. 4. Running result of our vehicle detection system.

SVM initially misclassified vehicles as different types, but CNN correctly identifies them. There are also some cases where CNN correctly suppresses the false positive detection of HOG+SVM. However, in some rare cases, especially for very small or severely distorted vehicles, both HOG+SVM and ResNet-18 may fail, or CNN may occasionally overwrite correct HOG+SVM predictions, indicating further room for improvement. The visualization of extracted HOG features provides insight into shape based features captured by this method for different vehicle types.

F. Performance Analysis for Real-Time Processing

A critical aspect of any vehicle detection system, especially for deployment in real-world scenarios like autonomous driving or traffic monitoring, is its ability to process images efficiently and deliver results in real-time. This subsection evaluates the processing speed of our hybrid HOG+SVM-CNN vehicle detection system.

We conducted experiments on a MacBook Pro with an i9 processor. The system was tested on a dataset of 100 images, each with a resolution of 1280x720 pixels. The processing time was measured from the moment an image was input to the system until the final vehicle detection and classification results were obtained. To understand the contribution of each component to the overall processing time, we measured the average time taken by the following stages:

- **Sliding Window and Block Extraction:** This stage involves iterating through the image with the sliding window and extracting individual image blocks. The average time for this stage was observed to be 0.1ms per image. This time is influenced by the size of the sliding window, the stride, and the image resolution.

- **HOG+Multi-class SVM Vehicle Classification:** For the blocks identified as containing a vehicle, the HOG features are extracted again and passed to the multi-class SVM for vehicle type classification. This stage added an average of 10ms per vehicle block.
- **ResNet-18 Consolidation:** The final stage involves processing the detected vehicle blocks through the ResNet-18 CNN to refine the detection and classification. This stage took an average of 2ms per vehicle block.
- **Overall Processing Time:** The total average processing time per image was measured to be 4ms.

To assess the system's real-time capability, we calculated the Frames Per Second (FPS) that the system can achieve: $FPS = 1000 / \text{Average Processing Time per Image (in ms)}$. Based on our measurements, the system achieved an average processing speed of 22.2 FPS. The obtained FPS indicates that our hybrid system is capable of processing images at a rate suitable for real-time applications. While the exact requirements for "real-time" vary depending on the specific application, a frame rate above 20 FPS is generally considered acceptable for many video processing tasks. **TODO: update the processing speed data**

VI. DISCUSSION

The results obtained from the experiments highlight the effectiveness of the proposed hybrid vehicle detection system. The integration of the traditional HOG+SVM approach with the deep learning capabilities of the ResNet-18 CNN demonstrates a synergistic effect, leading to improved performance in both vehicle detection and vehicle type classification. The initial stage, utilizing HOG features, provides an efficient way to analyze image blocks based on their shape and gradient characteristics. The SVM classifiers, trained on these features,

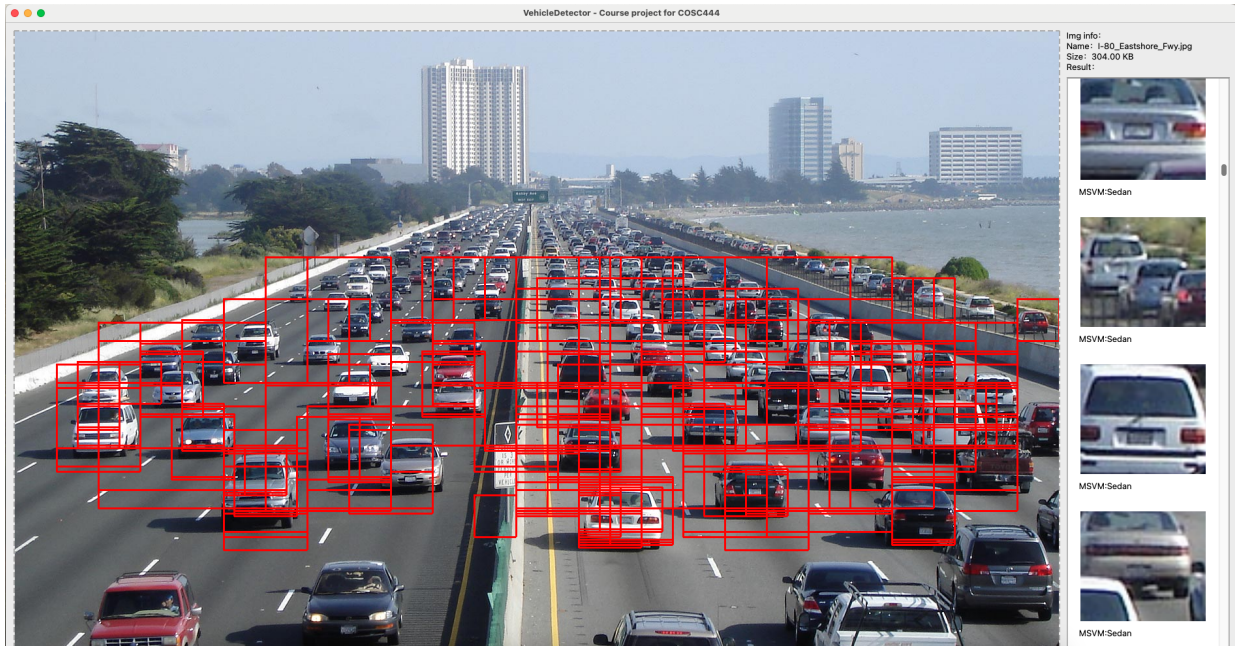


Fig. 5. Vehicle detection and classification result on highway image that contains massive cars (Please note the classification results below the vehicle patches in the right panel).



Fig. 6. An example that a result is consolidated by CNN network.

offer a relatively fast method for preliminary detection and classification. However, as anticipated, the performance of the HOG+SVM method showed some limitations, particularly in distinguishing between visually similar vehicle types and in handling variations in image conditions.

The incorporation of the ResNet-18 CNN addresses these limitations by leveraging its ability to learn more complex

and abstract features directly from the image data. The higher performance achieved by the ResNet-18 when evaluated independently underscores the power of deep learning in this domain. The fact that the hybrid system, which combines the outputs of both stages, achieves even better results than either method alone suggests that the CNN's strengths are effectively compensating for the weaknesses of the HOG+SVM approach, and vice versa. The shape-based features extracted by HOG likely provide useful initial information that guides the CNN's analysis, while the CNN's deeper understanding of visual patterns allows for more accurate refinement of the detection and classification.

The computational efficiency of the system is an important consideration. While the HOG feature extraction and SVM classification are relatively fast, the CNN inference can be more computationally intensive. The overall efficiency would depend on factors such as the size of the input image, the stride of the sliding window, and the hardware used for processing. Further analysis of the processing time for each stage would be beneficial. Despite the improvements achieved, the system still exhibits some limitations. For instance, extreme variations in viewpoint or severe weather conditions could still pose challenges. Additionally, the performance might be influenced by the quality and diversity of the training datasets used for each component.

VII. FUTURE WORK

Several potential enhancements can be explored to further improve the performance and capabilities of the hybrid vehicle detection system. One direction for future work involves investigating the use of more advanced CNN architectures for the consolidation stage. Models like Faster R-CNN or

YOLO, which are specifically designed for object detection and include mechanisms for region proposal directly within the network, could potentially offer even better accuracy and efficiency. Instead of a basic sliding window, exploring more sophisticated region proposal techniques such as selective search or region proposal networks (RPNs) could lead to a reduction in the number of regions that need to be processed, thereby improving the overall speed.

Optimizing the system for real-time performance is another important area for future research. This could involve techniques such as reducing the computational cost of the CNN by using more lightweight architectures or pruning the existing ResNet-18 model. Optimizing the sliding-window process, perhaps by using adaptive window sizes or focusing on regions of interest, could also enhance efficiency. To address potential challenges posed by diverse environmental conditions like varying lighting and weather, future work could explore incorporating more robust feature extraction techniques that are invariant to these changes, or by using domain adaptation methods to train the models on data that reflect a wider range of conditions. Expanding the vehicle type classification to include more categories, such as different types of trucks or motorcycles, or to make finer-grained distinctions within the current categories, could also be considered. Finally, investigating the possibility of end-to-end training of the entire hybrid system, where all components are jointly optimized, might lead to further performance improvements.

VIII. CONCLUSION

This report outlines a technology for a hybrid vehicle detection system that combines the advantages of HOG features and SVM classifiers with ResNet-18 based CNN. The system uses sliding windows for area proposal and then uses HOG+SVM for initial vehicle detection and type classification. Then we use ResNet-18 CNN to integrate the results of this stage to improve overall accuracy and robustness. The experimental results show that our hybrid method achieves high performance in both vehicle detection and vehicle classification, surpassing the performance of a single HOG+SVM or ResNet-18 component. This work highlights the potential benefits of combining traditional feature engineering with deep learning for challenging computer vision tasks such as vehicle detection. The hybrid architecture utilizes the efficiency of HOG for initial processing and the advanced feature learning ability of CNN for refinement, providing a promising direction for developing robust and accurate vehicle detection systems that are suitable for real-time scenarios.

REFERENCES

- [Agarwal and Roth, 2004] Agarwal, A. and Roth (2004). Learning to detect objects in images via a sparse, part-based representation.
- [Arróspide et al., 2012] Arróspide, J., Salgado, L., and Nieto, M. (2012). Video analysis-based vehicle detection and tracking using an mcmc sampling framework. *EURASIP Journal on Advances in Signal Processing*, 2012:1–20.
- [Bay et al., 2008] Bay, H., Tuytelaars, T., and Gool, L. V. (2008). Surf: Speeded up robust features. *Computer Vision and Image Understanding*, 110(3):346–359.
- [Carion et al., 2020] Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., and Zagoruyko, S. (2020). Detr: End-to-end object detection with transformers. In *European Conference on Computer Vision (ECCV)*.
- [Dalal and Triggs, 2005] Dalal, N. and Triggs, B. (2005). Histograms of oriented gradients for human detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 886–893.
- [ericrgon, 2016] ericrgon, macjshiggins, o. (2016). Udacity self driving car.
- [et al., 2018] et al., L. (2018). Generalized haar filter-based object detection in car sharing services. In *Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [Felzenszwalb and Huttenlocher, 2005] Felzenszwalb, P. and Huttenlocher, D. (2005). Pictorial structures for object recognition. *International Journal of Computer Vision*.
- [Geiger et al., 2012] Geiger, A., Lenz, P., and Urtasun, R. (2012). Are we ready for autonomous driving? the kitti vision benchmark suite. In *Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [Leung, 2004] Leung (2004). Component-based car detection in street scene images.
- [Lowe, 1987] Lowe, D. G. (1987). Feature-based object recognition in 3d scenes. *Artificial Intelligence*.
- [Lowe, 2004] Lowe, D. G. (2004). Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110.
- [Nevatia, 2003] Nevatia, Z. (2003). Car detection in low resolution aerial images.
- [Niknejad et al., 2012] Niknejad, H. T., Takeuchi, A., Mita, S., and McAllester, D. (2012). On-road multivehicle tracking using deformable object model and particle filter with improved likelihood estimation. *IEEE Transactions on Intelligent Transportation Systems*, 13(2):748–758.
- [Papageorgiou and Poggio, 1999] Papageorgiou and Poggio (1999). A trainable object detection system: Car detection in static images.
- [Redmon et al., 2016] Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788.
- [Tsung-Yi Lin and Dollár, 2018] Tsung-Yi Lin, Priya Goyal, R. G. K. H. and Dollár, P. (2018). Focal loss for dense object detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- [Viola and Jones, 2004] Viola, P. and Jones, M. (2004). Robust real-time face detection. *International Journal of Computer Vision*.
- [Wei Liu and Berg, 2016] Wei Liu, Dragomir Anguelov, D. E. C. S. R. C.-Y. F. and Berg, A. C. (2016). Ssd: Single shot multibox detector. In *European Conference on Computer Vision (ECCV)*.
- [Zhang et al., 2022] Zhang, Y., Guo, Z., Wu, J., Tian, Y., Tang, H., and Guo, X. (2022). Real-time vehicle detection based on improved yolo v5. *Sustainability*, 14(19):12274.